

**Lab Manual – 5**

Code/Course: 23CSE312/Distributed Systems

Topic: Remote Procedure Call (RPC)**[CO 05] [BTL -3]****StudentName:** G.Tejaswini**RollNo.:**23219**Batch/Section:** C**Date:** 02/03/2026**1 Introduction**

Remote Procedure Call (RPC) is a communication mechanism that enables a program to execute a procedure (function) on a remote machine as if it were a local function call. RPC allows a program running on one computer to call a function located on another computer without explicitly writing network communication code. It abstracts the complexities of:

- Socket programming
- Message formatting
- Data serialization
- Network transmission

Aim

To implement a simple Remote Procedure Call (RPC) service using rpcgen for an **addition** operation, compile server and client stubs, and execute a client-server RPC interaction over TCP. :contentReference[oaicite:2]index=2

Learning Outcomes

After completing this lab, you will be able to:

- Describe the role of an Interface Definition / specification file (.x) in ONC-RPC.
- Generate RPC stubs using rpcgen.
- Implement the server-side procedure `add` `proc 1 svc()`.
- Compile and run an RPC server and client and verify output.

Prerequisites / System Requirements

- **OS:** Linux/Unix-based OS (e.g., Ubuntu/Fedora/Solaris).
- **RPC Support:** Sun/ONC-RPC libraries available via libtirpc / rpcbind.

Installation (Ubuntu/Debian)

Run the following commands:

```
sudo apt-get install build-essential libtirpc -dev rpcbind sudo apt install  
rpcbind sudo service rpcbind start # Check RPC services rpcinfo
```

Lab Tasks (Step-by-Step)

Task 1: Create the RPC Specification File (add.x)

Create a file named add.x with the following contents.

```
struct add_in {  
    long arg1 ; long  
    arg2 ;  
};  
typedef long add_out ;  
  
program ADDPROC {  
    version ADD_VERS {  
        add_out ADDPROC( add_in ) = 1;  
    } = 1;  
} = 0x3543000 ;
```

Task 2: Generate Client/Server Stubs Using rpcgen

Generate the required files using:

```
rpcgen add . x
```

This generates (among others):

- add.h (header)
- add_svc.c (server stub)
- add_clnt.c (client stub)
- add_xdr.c (XDR marshalling/unmarshalling)

Task 3: Implement the Server Procedure (serverproc.c)

Create serverproc.c and implement the service function.

```
#include "add.h"
```

```

add_out *add proc 1 svc( add_in *in , struct svc_req *rqstp ) {
    static add_out out ; out = in->arg1 +
    in->arg2 ; return (&out );
}

```

Task 4: Compile and Run the Server

Compile and link the generated server files along with your procedure.

```

gcc -c serverproc . c gcc -c addsvc . c gcc -c add_xdr . c gcc -o server
serverproc . o add_svc . o add_xdr . o -lnsl
./ server

```

Task 5: Implement the Client (client.c)

Create client.c as below.

```

#include "add.h"
#include <stdio.h>
#include <stdlib.h>

int main( int argc, char **argv ) { CLIENT * cl
    ; add_in in ; add_out *out ;

    if ( argc != 4) {
        printf ("Usage : client <machine> <int1> <int2 >\n\n" );
        return 1;
    }

    cl = clnt_create ( argv [1] , ADD_PROG, ADD_VERS, "tcp" ); in . arg1 = atol (
    argv [ 2 ] ) ; in . arg2 = atol ( argv [ 3 ] ) ;

    out = add_proc 1(&in , cl);
    if ( out == NULL) {
        printf ("Error : %s\n", clnt_strerror ( cl , argv [ 1 ] ) ) ;
    } else {
        printf ("We received the result : %ld\n", *out );
    }

    clnt_destroy ( cl );
    return 0;
}

```

Task 6: Compile the Client

Compile and link the client with generated stubs and XDR functions.

```
gcc -c client.c gcc -c addclnt.c gcc -c add_xdr.c gcc -o client client
.o add_clnt.o add_xdr.o -lnsl
```

Task 7: Execute and Verify Output

1. Start the server in one terminal:

```
./server
```

2. Run the client in another terminal:

```
./client <server_address> 8 34 # Example :
./client      127.0.0.1 8 34
```

Expected output:

We received the result : 42

Notes

- The server must be running before the client sends a request.
- This example uses TCP ("tcp" in clnt create).
- Ensure the server IP/hostname passed to the client

Optional Extension: Concurrent RPC Server (Reading/Exploration)

By default, generated RPC servers are **iterative** (handle one request at a time). You may explore converting the server into a **concurrent server** using *threads* or *forking* to serve multiple clients simultaneously.

Observations/Output Screenshots

```
student@LAPTOP-PE7HTA21:~/rpc_lab$ gcc -c serverproc.c -I/usr/include/tirpc
student@LAPTOP-PE7HTA21:~/rpc_lab$ gcc -c add_svc.c -I/usr/include/tirpc
student@LAPTOP-PE7HTA21:~/rpc_lab$ gcc -c add_xdr.c -I/usr/include/tirpc
student@LAPTOP-PE7HTA21:~/rpc_lab$ gcc -o server serverproc.o add_svc.o add_xdr.o -ltirpc
student@LAPTOP-PE7HTA21:~/rpc_lab$ ./server
```

```
student@LAPTOP-PE7HTA21:~/rpc_lab$ gcc -c client.c -I/usr/include/tirpc
student@LAPTOP-PE7HTA21:~/rpc_lab$ gcc -c add_clnt.c -I/usr/include/tirpc
student@LAPTOP-PE7HTA21:~/rpc_lab$ gcc -c add_xdr.c -I/usr/include/tirpc
student@LAPTOP-PE7HTA21:~/rpc_lab$ gcc -o client client.o add_clnt.o add_xdr.o -ltirpc
student@LAPTOP-PE7HTA21:~/rpc_lab$ ./client 127.0.0.1 8 34
Result received: 42
student@LAPTOP-PE7HTA21:~/rpc_lab$ |
```

Submission Checklist

Submit the following:

- add.x

- serverproc.c
- client.c
- Terminal output screenshot(s) showing server start and client result
- Brief answers to viva questions in notebook